



Tarea módulo 1

Alejandro Sánchez Nogales



Índice

1. Análisis teórico de conceptos	3
1.1 Explicación de los conceptos básicos	3
1.2 Clasificación de lenguajes de programación	6
2. Actividad práctica y de análisis	9
2.1 Identificación de paradigmas de programación a partir de ejemplos	9
2.2 Actividad individual: imperativo vs declarativo	10



1. Análisis teórico de conceptos

1.1 Explicación de los conceptos básicos

Código fuente: Es un conjunto de instrucciones escritas por un informático que le dice a un programa o a una página web exactamente cómo debe funcionar. Es la versión de un software donde los humanos pueden leer, escribir y modificar.

Código objeto: Es el resultado de traducir el código fuente para que la máquina pueda empezar a entenderlo. Es el paso intermedio entre lo que escribe el programador y el programa final que se ejecuta en tu ordenador.

Código ejecutable: Es el producto final y definitivo del proceso de programación. Es el programa informático empaquetado y completamente listo para que tu ordenador (o teléfono) lo entienda y funcione correctamente pulsando doble clic.

- **Explica las fases por las que pasa un programa desde que es escrito en un lenguaje de programación hasta que es ejecutado en el procesador:**

El viaje desde que escribes una línea de código hasta que tu ordenador la ejecuta es un proceso estricto de traducción y ensamblaje. Tu procesador (la CPU) solo entiende señales eléctricas representadas en ceros y unos, por lo que el texto humano debe transformarse por completo.



- **Usa ejemplos concretos para cada concepto, mencionando en qué fase interviene cada uno en el desarrollo de un programa (análisis léxico, sintáctico, semántico, etc.).**

1. Análisis Léxico (El "Deletreador"):

El compilador lee el texto como una cadena de caracteres continuos y los agrupa en palabras válidas. Descarta los espacios en blanco y los comentarios.

Lo que entra: `int total = precio + 50;`

Si hay un error léxico: Si escribes `int 2total = ...` el analizador falla, porque los identificadores no pueden empezar por un número.

2. Análisis Sintáctico (El "Gramático"):

Toma los pasos anteriores y comprueba si están en el orden correcto según las reglas (gramática) del lenguaje. El resultado es un Árbol de Sintaxis Abstracta (AST), que organiza el código de forma jerárquica.

3. Análisis Semántico (El "Lógico"):

Verifica que lo que has escrito, además de estar bien escrito gramaticalmente, tenga sentido. Principalmente revisa la coherencia de los tipos de datos (Type Checking).



4. Generación de Código Intermedio:

El compilador traduce el árbol semántico a un lenguaje que está a medio camino entre el código fuente y el código de máquina. No está atado a ninguna CPU específica.

Ejemplo de lo que hace: Podría generar algo similar al código de "tres direcciones": $t1 = \text{precio} + 50$ (guarda la suma en un registro temporal t1).
 $\text{total} = t1$ (asigna el valor del temporal a la variable real).

5. Optimización (El "Afinador"):

Busca cómo hacer que ese código intermedio sea más rápido o consuma menos memoria, sin cambiar el resultado final.

Ejemplo de lo que hace: Si el compilador detecta que precio siempre vale 10 y nunca cambia antes de esta línea, en lugar de decirle a la CPU que haga la suma en vivo, optimiza la instrucción directa a: $\text{total} = 60$ (ahorrando tiempo de cálculo).

6. Generación de Código (El "Traductor Final"):

Convierte el código intermedio (ya optimizado) en código ensamblador o código máquina puro, usando las instrucciones específicas del procesador destino (por ejemplo, para Intel x86 o ARM de un móvil).



1.2 Clasificación de lenguajes de programación

1. Clasificación por Nivel de Abstracción

El nivel de abstracción mide qué tantos detalles del hardware (como la gestión de memoria o los registros del procesador) se le ocultan al programador.

Nivel	Descripción	Ejemplos Principales
Bajo Nivel	Interactúan directamente con el hardware. Son extremadamente rápidos y eficientes, pero difíciles de leer y escribir para los humanos. Dependen completamente de la arquitectura de la máquina (procesador).	Lenguaje Máquina (código binario de 0s y 1s), Ensamblador (Assembly).
Medio Nivel	Tienen estructuras de control similares al lenguaje humano (bucles, condiciones), pero aún permiten manipulación directa a nivel de sistema, como la gestión manual de la memoria mediante punteros.	C, C++, Rust. (Nota: Técnicamente se consideran de alto nivel en la academia, pero en la práctica industrial se clasifican como medio nivel por su acceso directo al hardware).
Alto Nivel	Ocultan completamente el hardware subyacente. Utilizan una sintaxis muy cercana al lenguaje natural (generalmente en inglés) y matemáticas. Suelen tener gestión automática de memoria (Recolector de basura).	Python, Java, JavaScript, C#, Ruby, PHP.

2. Clasificación por Paradigma de Programación

El paradigma es el "estilo" o la metodología que adopta el lenguaje para resolver un problema y estructurar el código.



Paradigma	Enfoque Principal	Subcategorías comunes	Ejemplos
Imperativo	Se centra en el CÓMO. El programador escribe una secuencia de instrucciones paso a paso que cambian el estado del programa hasta alcanzar el resultado.	Estructurada / Procedimental: Código dividido en bloques y funciones (Ej: C, Pascal). Orientada a Objetos (POO): Modela el problema usando "objetos" que contienen datos y métodos (Ej: Java, C++).	C, Java, Python, C++, Go.
Declarativo	Se centra en el QUÉ. El programador describe el resultado que desea obtener, y el lenguaje (o su motor subyacente) decide cómo calcularlo paso a paso.	Funcional: Basado en la evaluación de funciones matemáticas puras sin cambiar el estado (Ej: Haskell, Lisp). Lógico: Basado en reglas y hechos matemáticos (Ej: Prolog). Bases de Datos: Consultas directas de información (Ej: SQL).	SQL, Haskell, Prolog, HTML (<i>como lenguaje de marcado</i>).



Ejemplos de lenguajes para cada categoría:

Bajo Nivel

- **Ensamblador (Assembly):** Utiliza comandos de texto simples (como ADD o MOV) que se mapean uno a uno con las instrucciones físicas exactas del procesador. El programador debe gestionar los registros internos de la CPU manualmente.
- **Lenguaje Máquina:** Es el código nativo binario (0s y 1s) o hexadecimal. Pertenecer a esta categoría porque es el único lenguaje que el hardware lee y ejecuta de forma directa, sin ninguna capa intermedia de traducción.

Medio Nivel

- **C:** Permite programar usando bucles, condicionales y funciones estructuradas (rasgos de alto nivel), pero proporciona "punteros", una herramienta que permite al programador manipular direcciones de memoria física del sistema operativo directamente.
- **Rust:** Ofrece una sintaxis moderna y comprobaciones de seguridad estrictas antes de compilar el código, pero al igual que C, no tiene un recolector de basura automático, otorgando al programador un control milimétrico sobre cómo se asigna y libera la memoria en el hardware.

Alto Nivel

- **Python:** Su sintaxis prioriza la legibilidad humana, pareciéndose mucho al inglés escrito. Oculta por completo el hardware: el programador no necesita asignar memoria para una variable, ya que el lenguaje lo hace y lo limpia automáticamente (mediante su *garbage collector*).
- **Java:** Abstrae no solo la memoria, sino el propio sistema operativo. Su código se compila para ejecutarse dentro de la Máquina Virtual de Java (JVM), lo que significa que el programa interactúa con un entorno de software estandarizado en lugar de interactuar directamente con el hardware real.



2. Actividad práctica y de análisis

2.1 Identificación de paradigmas de programación a partir de ejemplos

Fragmento 1 — Imperativo

El programa describe la secuencia de pasos a seguir: recorre la lista elemento por elemento hasta acumular la suma. Se detalla cómo se llega al resultado (bucle, variable acumuladora, iteración), característica típica del paradigma imperativo.

Fragmento 2 — Declarativo

La consulta especifica qué se quiere obtener (nombres de empleados mayores de 30 años) sin indicar los pasos internos para lograrlo (no se dice cómo recorrer la tabla, qué estructura de control usar, etc.). Esto pertenece al paradigma declarativo.

Fragmento 3 — Declarativo

Aunque el factorial pueda ponerse de forma imperativa, aquí se puede ver una relación matemática sin especificar los pasos concretos (no hay bucles ni variables de control explícitas). Esta forma de definir el problema mediante reglas o funciones matemáticas es característica del paradigma declarativo.

Fragmento 4 — Imperativo

Se describe el proceso paso a paso: recorrer la lista y comprobar cada producto individualmente para decidir si cumple la condición del precio. Al detallar el cómo se realiza el filtrado (iteración explícita, comprobación uno por uno), corresponde al paradigma imperativo.



2.2 Actividad individual: imperativo vs declarativo

Actividad elegida: Preparar café con leche por la mañana

Descripción Imperativa

1. Llenar la cafetera con 200 ml de agua.
2. Poner 2 cucharadas de café molido en el filtro.
3. Encender la cafetera y esperar a que termine (unos 5 minutos).
4. Verter la leche en un cazo.
5. Calentar la leche a fuego medio durante 2 minutos, removiendo para que no se pegue.
6. Retirar la leche del fuego justo antes de que hierva.
7. Coger una taza.
8. Verter el café en la taza hasta la mitad.
9. Añadir la leche caliente hasta llenar la taza.
10. Añadir azúcar al gusto y remover con una cuchara.

Descripción Declarativa

"Quiero una taza de café con leche caliente, con el punto de azúcar a mi gusto, y lista para tomar."

Ventajas y desventajas:

Enfoque imperativo

- Control total del proceso, resultado reproducible.
- Descripción larga y rígida ante cambios o imprevistos.



Enfoque declarativo

- Descripción breve y flexible, permite adaptar el método.
- Requiere conocimiento previo de quien ejecuta y no garantiza el mismo resultado siempre.